

Практическая работа Тема 4. Работа с выводами общего назначения.

Цель: познакомиться с понятием «выводы общего назначения», получить практические навыки по устройству и настройке параллельного порта, и выводов общего назначения в МК AVR, закрепить на практике навыки программирования на языке Assembler и C++.

Используемое оборудование

Компьютер с ОС Windows 1 шт.

Отладочная плата Arduino UNO 1 шт.

Программатор 1 шт.

Логический анализатор 1 шт.

Кнопочный модуль 1 шт.

Используемое ПО: Интегрированная среда разработки Atmel Studio 7.0 (или AVR Studio 4.19) Программа для загрузки программного кода в микроконтроллер AVRDUDEPROG

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Выводы общего назначения

Для обмена информацией с внешними устройствами микроконтроллер имеет набор выводов. Количество этих выводов у разных контроллеров сильно отличается и влияет на то, сколько устройств можно подключить к МК одновременно. Обычно количество информационных и управляющих линий устройств, входящих в состав микроконтроллера, превышает количество физических выводов МК. Поэтому физические выводы могут быть использованы в разных режимах. Их называют **выводами общего назначения – GPIO** (General Pin In Out). Однако основным режимом, как правило, является режим цифрового вывода или при использовании совокупности выводов - режим параллельного порта.

Средства обмена данными, а именно линии передачи, устройство управления передачей и протокол обмена называются **интерфейсом передачи данных**. Устройство управления передачей называют **контроллером интерфейса периферийных устройств**. Интерфейсы в свою очередь делятся на параллельные и последовательные. Параллельные интерфейсы могут передавать одно слово данных по нескольким линиям одновременно. Например, параллельные порты МК AVR – 8-ми битные. Однако чаще параллельные порты используются для опроса простых датчиков, возвращающих только значение 1/0, подключения кнопок, а также элементов индикации и управления устройствами через реле, т.е. используются отдельные однобитные выводы портов.

Параллельные порты микроконтроллеров AVR

За конфигурацию параллельного порта в микроконтроллерах AVR отвечают тройки 8-битных регистров: PORTx, DDRx, PINx, где x – имя порта:

PIN_x – регистр ввода данных. В нем содержится информация о текущем состоянии выводов МК, относящихся к этому порту. Регистр доступен только в режиме чтения.

DDR_x – определяет направление работы порта (ввод или вывод данных). DDR_{xy}=0 – вывод работает как ВХОД. DDR_{xy}=1 вывод работает на ВЫХОД, где X – имя порта, Y – номер вывода (0-7). Регистр позволяет задавать отдельные выводы порта в любом режиме.

PORT_x – назначение регистра зависит от состояния DDR_x. Если вывод порта настроен на передачу данных, то в регистр PORT_x записывается значение, которое необходимо вывести. Если вывод настроен на получение, то PORT_x определяет, используется ли на выводе подтяжка вверх (PULL UP).

Подтяжкой вверх называется соединение вывода микросхемы с линией питания через резистор. Такой режим гарантирует, что в случае отключения внешнего устройства или в случае неустойчивого сигнала, на входе микроконтроллера будет устойчивое состояние 1.

В микроконтроллере AVR ATmega328P 3 параллельных порта: В,С,Д, каждый из которых управляется своей тройкой регистров.

Сводная таблица описанных регистров параллельных портов представлена ниже.

Адрес в ОЗУ	Адрес в I/O Reg	Имя регистра	Назначение битов регистра [7:0]							
			7	6	5	4	3	2	1	0
0x23	0x03	PINB	PINB7	PINB6	PINB5	PINB4	PINB3	PINB2	PINB1	PINB0
0x24	0x04	DDRB	DDRB7	DDRB6	DDRB5	DDRB4	DDRB3	DDRB2	DDRB1	DDRB0
0x25	0x05	PORTB	PORTB7	PORTB6	PORTB5	PORTB4	PORTB3	PORTB2	PORTB1	PORTB0
0x26	0x06	PINC		PINC6	PINC5	PINC4	PINC3	PINC2	PINC1	PINC0
0x27	0x07	DDRC		DDRC6	DDRC5	DDRC4	DDRC3	DDRC2	DDRC1	DDRC0
0x28	0x08	PORTC		PORTC6	PORTC5	PORTC4	PORTC3	PORTC2	PORTC1	PORTC0
0x29	0x09	PIND	PIND7	PIND6	PIND5	PIND4	PIND3	PIND2	PIND1	PIND0
0x2A	0x0A	DDRD	DDRD7	DDRD6	DDRD5	DDRD4	DDRD3	DDRD2	DDRD1	DDRD0
0x2B	0x0B	PORTD	PORTD7	PORTD6	PORTD5	PORTD4	PORTD3	PORTD2	PORTD1	PORTD0

Адрес в ОЗУ – реальное расположение регистра в адресном пространстве оперативной памяти. Используя этот адрес, можно обращаться к регистрам с помощью команды ST, LD, STS, LDS.

Адрес в I/O Reg – смещение адреса регистра относительно сегмента I/O Registers (начинается с адреса 0x20 – ОЗУ). Используя этот адрес можно обращаться к регистрам с помощью команды IN/OUT.

Имена регистров в данной группе соответствуют адресам I/O Reg, их можно использовать в командах IN/OUT.

Команды управления регистрами периферийных устройств

Регистры PIN_x, DDR_x, PORT_x относятся к сегменту регистров периферийных устройств. Для работы с этим сегментом используются следующие команды:

- out <ioreg>, <reg> – запись значения регистра <reg> в регистр устройств ввода / вывода <ioreg>;

- `in <reg>, <ioreg>` – загрузка состояния регистра устройств ввода / вывода `<ioreg>` в регистр общего назначения `<reg>`;
- `sbi <ioreg>, <num>` – устанавливает бит № `<num>` регистра `<ioreg>` в единицу (`num` – может принимать значение 0-7, где 0 младший бит регистра, 7 старший);
- `cbi <ioreg>, <num>` – сбрасывает бит № `<num>` регистра `<ioreg>` в ноль (`num` – может принимать значение 0-7, где 0 младший бит регистра, 7 старший).

Обращение к регистрам ПУ на языке C

Для работы с регистрами периферийных устройств, в стандартных библиотеках AVR «`io.h`» описаны идентификаторы портов. Имена идентификаторов соответствуют именам регистров. Помимо этого, в библиотеках описаны константы, соответствующие номерам битов в регистре. Например, `PORTD6` – константа, соответствующая числу 6.

!Пример может показаться избыточным, и в случае с параллельным портом оно так и есть, однако в большинстве регистров ПУ каждый бит выполняет свою функцию и имеет свое имя, поэтому удобно не запоминать положение бита в регистре, а лишь запомнить его имя.

Для того, чтобы записать значение в регистр, достаточно выполнить операцию присвоения, например `PORTD = 0x12`. Считать значение в переменную можно, выполнив аналогичную операцию `int x = PORTD`.

Битовые операции удобно выполнять, используя логические операции с масками.

Установка отдельных битов: `PORTD = PORTD | 0x12`,

или `PORTD |= 0x12`,

или `PORTD |= (1<<4) | (1<<1)`,

или используя константы, обозначающие номера конкретных битов регистра, `PORTD |= (1<<PORTD4) | (1<<PORTD1)`.

Сброс отдельных битов: `PORTD = PORTD & 0xED`,

или `PORTD = PORTD & ~0x12` `PORTD &= ~0x12`,

или `PORTD &= ~(1<<4) | (1<<1)`,

или `PORTD &= ~(1<<PORTD4) | (1<<PORTD1)`.

Инверсия отдельных битов: `PORTD = PORTD ^ 0x12`,

или `PORTD ^= ~0x12`,

или `PORTD ^= (1<<4) | (1<<1)`,

или `PORTD ^= (1<<PORTD4) | (1<<PORTD1)`.

Чтение и анализ битов также удобно выполнять с помощью логических операций и масок.

Отладка и анализ работы ПУ в IDE Atmel Studio

При отладке работы микроконтроллера можно анализировать изменения, происходящие в регистрах ПУ. Для этого после входа в режим отладки необходимо выбрать Debug/Windows/IO. Открывшееся окно (рис. 4.1) состоит из 2 частей. В верхней части окна можно найти интересующее периферийное устройство, выбрав из списка или указав имя в поле Filter. В нижней части показаны регистры выбранного устройства и значения, записанные в них: закрашенный квадратик – «логическая единица», не закрашенный – «логический ноль».

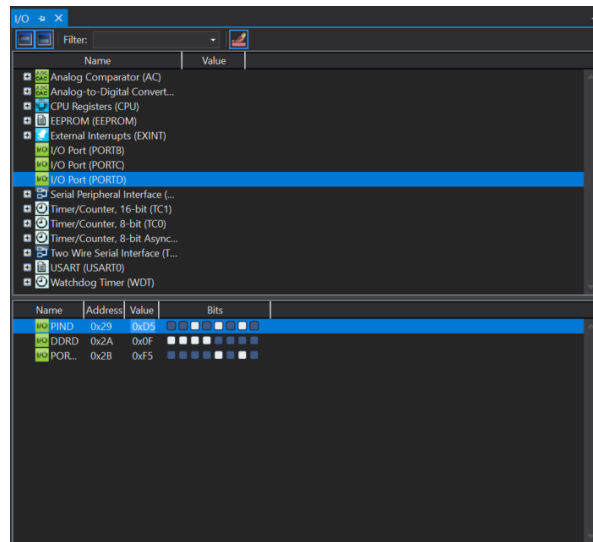


Рис. 4.1. Окно I/O в режиме Debug

Изменять значения битов регистров можно в процессе отладки, кликнув по интересующему биту. Таким образом, можно имитировать работу внешних устройств, устанавливая в определенные моменты времени различные значения.

ПРАКТИЧЕСКАЯ ЧАСТЬ

Вывод информации через параллельный порт

Пример 1. Рассмотрим использование параллельного порта микроконтроллера в режиме «вывода данных». Ниже представлен код программы, которая обеспечивает последовательный вывод чисел от 0 до 255 в порт D.

Пример 1 «Assembler»

1	start:	;метка начала программы
2		;настройка порта на вывод данных
3	ldi r16, 0xFF	;запись константы 0xFF в регистр r16
4	ldi r17, 0x00	;запись константы 0x00 в регистр r17
5	out DDRD,r16	;запись константы 0xFF из регистра r16 в регистр DDRD (режим
6		; вывод данных)
7	p:	;метка начала рабочего цикла (основной код программы)
8	out PORTD, r17	;вывод значения регистра r17 в порт d
9	inc r17	;инкремент регистра r17
10	rjmp p	;безусловный переход к метке p

Создайте новый проект. В параметрах выберите микроконтроллер ATmega328P. Наберите текст программы и выполните его пошаговую отладку в программе Atmel Studio (AVR Studio). Проанализируйте работу с использованием окна I/O.

Подключите программатор к отладочному стенду AVR. Подключите логический анализатор (устройство, позволяющее посмотреть логические уровни на выводах схемы во времени) к выводам порта D согласно схеме на рисунке 4.2. Запустите программу Logic (Пуск/Saleae Inc/Logic). Убедитесь, что анализатор подключен к устройству согласно схеме и подключен к компьютеру по интерфейсу USB. В строке заголовка программы Logic должна появиться надпись [Connected]. Выполните загрузку кода программы в микроконтроллер согласно инструкции см. приложение. В программе Logic нажмите Start Simulation. Посмотрите полученные диаграммы, проанализируйте значения, поясните результаты по коду программы

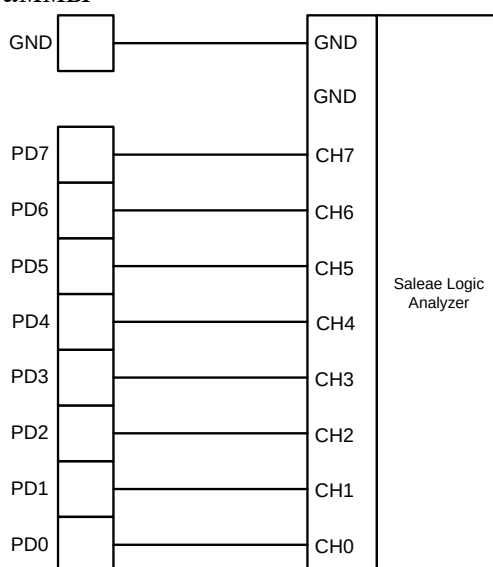


Рис. 4.2. Подключение логического анализатора к выводам порта D

Создайте проект для программы на языке C и повторите эксперимент с примером, предложенным ниже. Обратите внимание на поле Cycle Counter при отладке программ в среде программирования.

Пример 1 «C»

1	<code>#include <avr/io.h></code>	;подключение необходимых библиотек
2	<code>int main(void)</code>	;вход в программу
3	<code>{</code>	;инициализация параметров
4	<code>int x;</code>	;объявление переменной x
5	<code>DDRD = 0xFF;</code>	;настройка порта D на вывод информации
6	<code>while (1)</code>	;начало бесконечного цикла (основной код программы)
7	<code>{</code>	
8	<code>PORTD = x;</code>	;вывод значения регистра r17 в порт d
9	<code>x++;</code>	;инкремент значения x
10	<code>}</code>	
11	<code>}</code>	

Выполните анализ работы аналогично примеру на Assembler.

Задание 1. Измените код из примера 1 таким образом, чтобы при переполнении значения регистра r1 счетчик менял направление счета с +1 на -1, а при займе – с -1 на +1. Проанализируйте работу программы в отладчике. Анализатор подключите к выводам согласно рис. 4.2а. загрузите программу в контроллер. Проанализируйте поведение сигналов в программе Logic.

Задание 2. Измените код программы из задания 1 таким образом, чтобы на вывод работали только выводы PORTD6-PORTD7. Используя логический анализатор, докажите корректность работы программы.

Пример 2. Рассмотрим программу, управляющую светодиодным индикатором (L – PB5):

Пример 2 «Assembler»

1	start:	;метка начала программы
2	ldi r16, 0x20	;запись константы 0x20 в регистр r16
3	out DDRB, r16	;запись константы из r16 в DDRB, определяет pin5 на вывод
4		;остальные биты инициализируются как входные
5	p:	;метка для организации бесконечного цикла
6	subi r17,0xff	;вычитание r17 числа -1 в ДК (операция inc)
7	adc r18, r20	;сложение регистра r18 с флагом с от прошлой операции
8		;(r18=r18+r20+c, где r20=0)
9	adc r19, r20	;сложение регистра r19 с флагом с от прошлой операции
10		;(r19=r19+r20+c, где r20=0)
11	bst r19,4	;анализ 4ого бита (отсчёт с 0) регистра r19 (20 бита слова
12		;хранящегося в r19,r18,r17)
13	BRTS yes	;переход если пользовательский бит=1
14	BRTC not	;переход если пользовательский бит=0
15	rjmp p	;возврат в бесконечный цикл
16	yes:	;метка начала процедуры, выполняемой, если пользовательский бит=1
17	sbi PORTB, 5	;установить 1 в pin0 порта B – включает светодиод L
18	rjmp p	;возврат в бесконечный цикл
19	not:	;метка начала процедуры, выполняемой, если пользовательский бит=0
20	cbi PORTB, 5	;установить 1 в pin0 порта B – включает светодиод L
21	rjmp p	;возврат в бесконечный цикл

Ниже представлена аналогичная программа на языке высокого уровня C.

Пример 2 «C»

1	#include <avr/io.h>	;подключение необходимых библиотек
2	uint32_t x=0;	;объявление переменной (счётчика) - x
3	int flag=0;	;флаг для анализа текущего состояния
4	int main(void)	;вход в программу
5	{	;инициализация параметров
6	DDRB = 0x20;	;настройка порта D (0) на вывод информации
7	while (1)	;начало бесконечного цикла (основной код программы)
8	{	
9	if (x==800000)	;если счётчик досчитал до 8000000
10	{if (flag==0)	;то, если flag = 0
11	{	
12	PORTB=0x20; flag=1;	;включение светодиода, флаг = 1
13	}	
14	else	;иначе (если флаг равен 1)
15	{	
16	PORTB=0x00; flag=0;	;выключение светодиода, флаг = 0
17	}	
18	x=0;}	;обнуление x
19	x++;	;инкремент значения x
20	}	
21	}	

Микроконтроллер работает под воздействием генератора с частотой 16 МГц. Очевидно, что если на такой частоте в программе будет только изменяться состояние светодиода, то его мигание станет незаметно глазу человека, диод будет всегда гореть. Поэтому в программе представлен пример анализа бит многоразрядного счетчика. Если 20 бит группы из трех регистров r19, r18, r17 (r17 – хранит младший байт) равен 1, то светодиод загорается, иначе – гаснет.

Загрузите программу в МК. Проанализируйте работу отладочной платы.

Задание 3. Измените код программы таким образом, чтобы частота мигания светодиода изменилась по следующему правилу:

Вариант	1	2	3	4	5	6	7	8	9	10
Задание	Clk*2	Clk/2	Clk*4	Clk/4	Clk*8	Clk/8	Clk/16	Clk/32	Clk/16	Clk/32

Ввод информации через параллельный порт

Пример 3. Обработка внешних сигналов. В качестве источника внешнего сигнала будем использовать кнопку, замыкающую вход микроконтроллера на землю (gnd). Рассмотрим код программы.

Пример 3 «Assembler»

1	start:	;инициализация порта B
2	ldi r16, 0x20	;запись константы 0x20 в регистр r16
3	out DDRB, r16	;запись константы 0x20 из r16 в DDRB – определяет pin0 на вывод
4		;остальные биты инициализируются как входные
5	ldi r16, 0x01	;запись константы 0x01 в регистр r16
6	out PORTB, r16	;запись константы 0x01 из r16 в PORTB – включает подтяжку PULL-UP
7		;на выводе pin 0 порта B
8	p:	;метка для организации бесконечного цикла
9	in r17, PINB	;чтение данных в r17 из регистра ввода данных PINB
10	bst r17,1	;загрузка младшего бита регистра r17 в бит пользователя
11	BRTS yes	;переход если пользовательский бит=1
12	BRTC not	;переход если пользовательский бит=0
13	rjmp p	;возврат в бесконечный цикл
14	yes:	;метка начала процедуры, выполняемой, если пользовательский бит=1
15	sbi PORTB, 5	;установить 1 в pin0 порта B – включает светодиод AT0
16	rjmp p	;возврат в бесконечный цикл
17	not:	;метка начала процедуры, выполняемой, если пользовательский бит=0
18	cbi PORTB, 5	;установить 1 в pin0 порта B – включает светодиод AT0
19	rjmp p	;возврат в бесконечный цикл

Ниже представлена аналогичная программа на языке высокого уровня C.

Пример 3 «C»

1	#include <avr/io.h>	;подключение необходимых библиотек
2	int main(void)	;вход в программу
3	{	;инициализация параметров
4	DDRB =0x20;	;настройка порта B (5) на вывод информации
5	PORTB = 0x01;	;настройка порта B (0) на ввод информации с «подтяжкой» вверх
6	while (1)	;начало бесконечного цикла (основной код программы)
7	{	
8	if(PINB&0x01)==0x00)	;если кнопка нажата
9	PORTB = 0x20;	;зажигается диод
10	else PORTB~=! 0x20;	;иначе, диод выключается
11	}	
12	}	

В программе вывод, к которому подключается кнопка, настраивается на прием данных в режиме PULL-UP. Это необходимо для того, чтобы в момент, когда кнопка разомкнута, микроконтроллер всегда имел на входе логическую единицу. Когда же кнопка замкнута, она напрямую (без сопротивления) соединяет ввод микроконтроллера и землю (логический ноль). Резистор, через который реализована подтяжка, имеет сопротивление, поэтому весь ток уходит на землю. На входе МК получаем значение логического 0.

Выполните сборку проекта. Проанализируйте работу программы в отладчике, генерируя нажатие на кнопку изменением состояния в соответствующем бите регистра PINху.

Отключите питание от отладочной платы. Подключите кнопочный модуль согласно схеме на рисунке 4.3. Подключите питание, загрузите программу в МК. Проанализируйте работу программы.

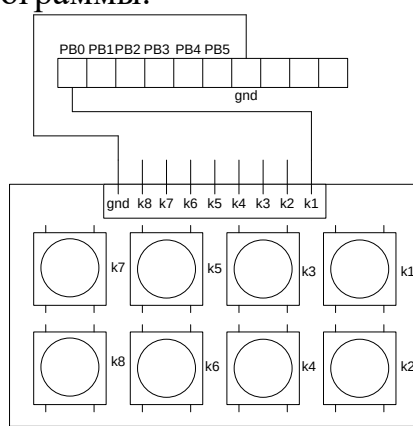


Рис. 4.3. Подключение кнопок к отладочной плате

Задание 4. Разработайте программу, которая по нажатию на кнопку 1 увеличивала бы, а по нажатию на кнопку 2 – уменьшала состояние регистра R0 на 1. Значение регистра R0 должно выводиться в порт D. Кнопки подключите к выводам (1 – PB0, 2 – PB1)

Выполните сборку программы. Проанализируйте её в отладчике, имитируя нажатия на кнопки.

Загрузите программу в МК, проанализируйте работу программы с помощью логического анализатора. Анализатор подключите согласно рисунку 4.2.

Задание 5. Разработайте программу на языке C, которая обеспечивала бы управление светодиодами (RX – PD0, TX – PD1, L – PB5) с помощью кнопок (кнопки подключите к выводам PD2, PD3, PD4). Нажатие на кнопку изменяет состояние светодиода на противоположное, после чего оно сохраняется.

Выполните сборку программы. Проанализируйте её в отладчике, имитируя нажатия на кнопки.

Подключите кнопки к плате в соответствии с заданием. Покажите подключение преподавателю. Загрузите программу в контроллер. Проанализируйте работу системы.

***Задание 6.** Попробуйте оптимизировать программы из примеров, представленных в работе.

В отчете представьте коды всех программ (включая примеры и задание 4) с комментариями, выводы по результатам выполнения программ, снимки экрана программы Logic (для примера 1 и заданий 1,2,4).

Контрольные вопросы

1. Что называют GPIO?
2. Как используется параллельный порт микроконтроллера?
3. Что называют «подтяжкой» вывода, и зачем она используется?
4. Какие регистры отвечают за настройку параллельного порта?
5. Сколько параллельных портов реализовано в МК ATmega328p?
6. Зачем нужны команды SBI и CBI?
7. Что называют интерфейсом передачи данных?
8. Что называют контроллером интерфейса периферийных устройств?
9. Опишите последовательность действий при загрузке программы в МК.
10. Опишите устройство отладочного комплекта AVR.